Whichever approach you took to scaffolding the CRUD pages, you should now have the same results. The first thing you may notice is that they fail to build. At the time of writing, there is a bug in the scaffolding tool that results in the `@` character (which cannot be used as a valid C# identifier) being applied as a parameter in the `Include` methods in the `Edit`, `Delete`, `Details`, and `Index PageModel` classes:

```
Property = await _context.Properties
    .Include(@ => @.City).FirstOrDefaultAsync(m => m.Id == id);
```

This needs to be replaced with another character—say, `p`:

```
Property = await _context.Properties
    .Include(p => p.City).FirstOrDefaultAsync(m => m.Id == id);
```

Once that change has been made, you can examine the code in the various pages and look to improve it. In this chapter, I focus on the files for the Edit page. Once you understand the range of issues that need to be addressed in this page, you will be in a good position to apply appropriate changes to the other pages.

I'll start by showing the first few lines of the `EditModel` code. The first thing you will notice is that `EditModel` takes a dependency on the EF Core context.

Listing 8.22 The scaffolded `EditModel` constructor

```
private readonly CityBreaks.Data.CityBreaksContext _context;

public EditModel(CityBreaks.Data.CityBreaksContext context)
{
    _context = context;
}
```